A $\mathcal{PS}$-complete problem

Just like in the class $\mathcal{NP}$, we know a $\mathcal{NP}$-complete language: the True Quantified Boolean Formulas (TQBF):

$$\text{TQBF} = \{\langle\phi\rangle \mid \phi \text{ is a true fully quantified boolean formula.}\}$$

A *fully quantified* boolean formula is a ϕ just like the ones in 3SAT, except every variable has a quantifier: e.g.,

$$\phi = \forall x, y, \exists z, (x \vee z) \wedge (\overline{y} \vee z).$$

The proof that this is $\mathcal{PS}$-complete is somewhat similar to the proof of the Cook-Levin theorem that we saw last week, with a space-saving mechanism a little bit like the one used in the proof of Savitch's theorem.

You can think of an $\mathcal{NP}$ problem like SAT or 3SAT as a problem in which there is only a single existential quantifier "$\exists$" that covers all of the variables. Similarly, co-$\mathcal{NP}$ uses a single universal quantifier "$\forall$". This is a generalization.

Why $\mathcal{PS}$ is Interesting

We can treat choices made under quantifiers as being made by adversaries in a game:

- If I say that there is an input x that makes something like a boolean formula ϕ true ("$\exists$"), I'm saying I could tell you that x, so long as I can find it.
- If I say that for all inputs y, ϕ true ("$\forall$"), I'm saying that you can't tell me an input y that makes that ϕ false.
- If I chain many quantifiers together, we can take turns choosing inputs – I choose the ones covered by"$\exists$" and you choose the ones covered by "$\forall$". If I say the ϕ is true with these quantifiers, what I'm saying is that even if you try to make the ϕ false, I can always make choices that force the end result to be true.

Why $\mathcal{PS}$ is Interesting (2)

- If we were playing a game in which I was trying to make ϕ true while you were trying to make it false, and if I could always manage this, I'd be saying that I had a winning strategy.
- When we look at it this way, $\mathcal{PS}$-complete problems start to look like games – we take turns making moves, and we each try to reach our (mutually exclusive) winning conditions. So it'll not be too much of a surprise that a number of generalizations of two-player games are $\mathcal{PS}$-hard.

This is one reason why $\mathcal{PS}$ is important — $\mathcal{PS}$-hardness comes up when we're working in areas in which a program (like a robot) is competing against another agent. It also comes up when the robot is repeatedly taking measurements of its surroundings – our worst-case analysis treats the surroundings like an adversary, even if there is no actual adversary present.

An Example $\mathcal{PS}$-comeplete Game

We've said that many generalizations of games are $\mathcal{PS}$-hard. These games include Hex, Reversi, and some versions of Go (when these games are played on arbitrarily large boards).

We should note that several one-player games such as Sokoban also have $\mathcal{PS}$-complete extensions.

To show these games are complete, we need to reduce to them from another $\mathcal{PS}$-complete problem. We'll give an example of one such reduction to a simple game: the generalized geography game.

Generalized Geography

When playing *Geography*, players take turns naming cities. There are two rules:

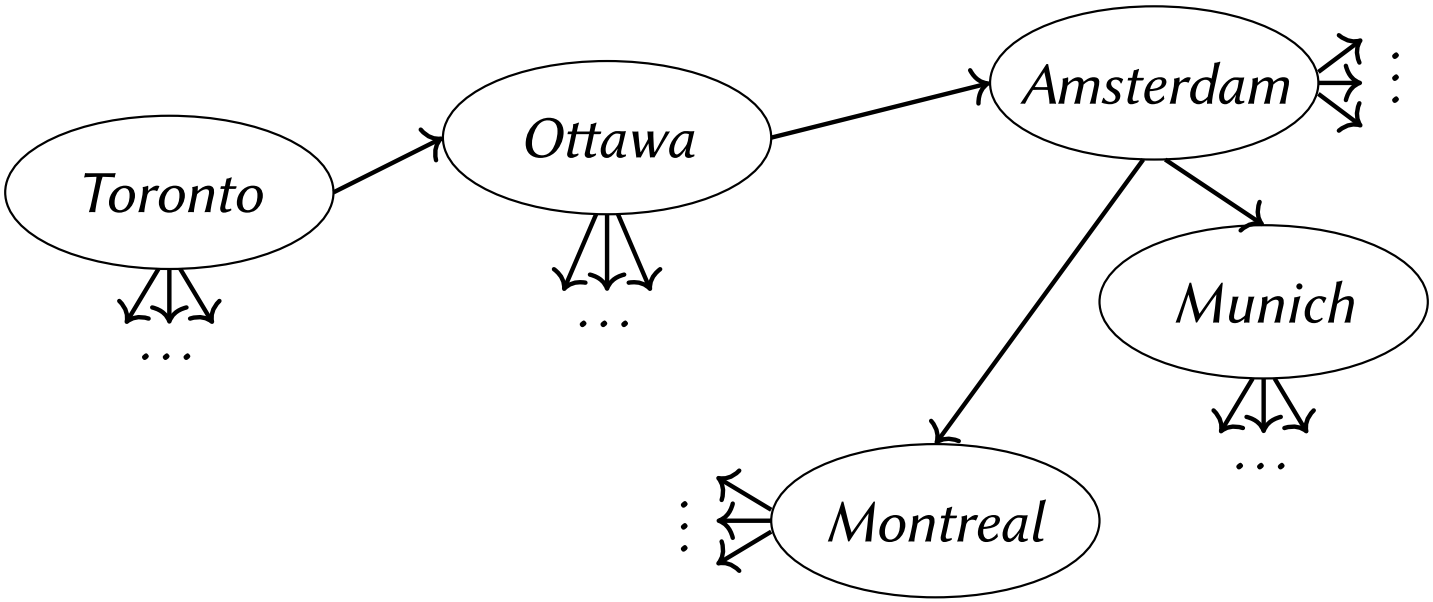
1. No city can be named more than once.
2. The last letter of the current city named must be the first letter of the next.

E.g., *Toronto, Ottawa, Amsterdam, Munich, ...*.

The first player who cannot continue the chain loses.

Generalized Geography (2)

We can express this game by putting different city names on a graph:



That is, we draw an arrow between nodes u and v if the last letter of the city name in u is the first letter in v.

Generalized Geography (3)

When using this graph to play geography, each player takes turns choosing the next node to visit: the next node must be reachable from the current one using a single edge, and no node can be visited twice.

Generalized Geography is the same, except we can play it on any directed graph G. In order to argue that this game is $\mathcal{PS}$-complete, we treat it as a language. So we formulate the game as:

$$\mathrm{GG} = \left\{ \langle G, s \rangle \;\middle|\; \begin{array}{l} \text{Player A has a winning strategy for generalized} \\ \text{geography using } s \text{ as the starting vertex.} \end{array} \right\}$$

Generalized Geography is in $\mathcal{PS}$

We can solve it using the following code:

$M =$ "On input $\langle G, s \rangle$:

1. If s has out-degree zero, reject, since player A loses.
2. Remove s and all edges containing s.
3. For each of the nodes $v_1, v_2, \ldots, v_k$ that s originally pointed to, recursively call M on $\langle G, v_i \rangle$.
4. If all recursive calls accept, player B has a winning strategy, so *reject*. Otherwise, player B doesn't have a winning strategy so player A must be able to force a win. So *accept*.

Question: How much space is required for this program?

There are at most ($n =$ the number of nodes in G) levels of recursion, and each step in the recursion just requires us to store a subgraph of G. So this is in $\mathcal{PS}$.

Generalized Geography is $\mathcal{PS}$-hard

So to show that this is $\mathcal{PS}$-complete, we just need to show that TQBF $\leqslant_p$ GG.

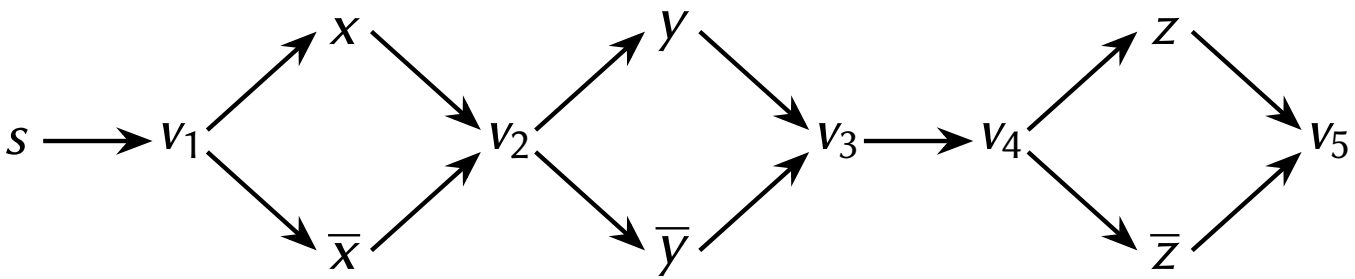
Recall that this means that we want to be able to use GG to solve TQBF – if we have a quantified boolean formula ϕ we'd like to build, in polynomial time, a corresponding graph G with a starting node s. If we do it right, then determining if G and s are in GG would tell us whether ϕ is in TQBF.

Let's have a look at the ϕ from the previous class:
$\phi = \forall x, y, \exists z, (x \vee z) \wedge (\overline{y} \vee z)$.

How do we Start?

Quantifiers are a bit like player choices – one player chooses the variables under the $\forall$ quantifiers and tries to make the formula false, while another chooses the variables under the $\exists$ quantifiers and tries to make it true. We say that player A is trying to make the formula true.

So player B chooses x and y, and player A chooses z. A good place to start building our full graph would be to build a graph that makes the players go through a similar choice. One possible way to do this is to create nodes x, $\overline{x}$, and so on, in the graph, and connect them like so:

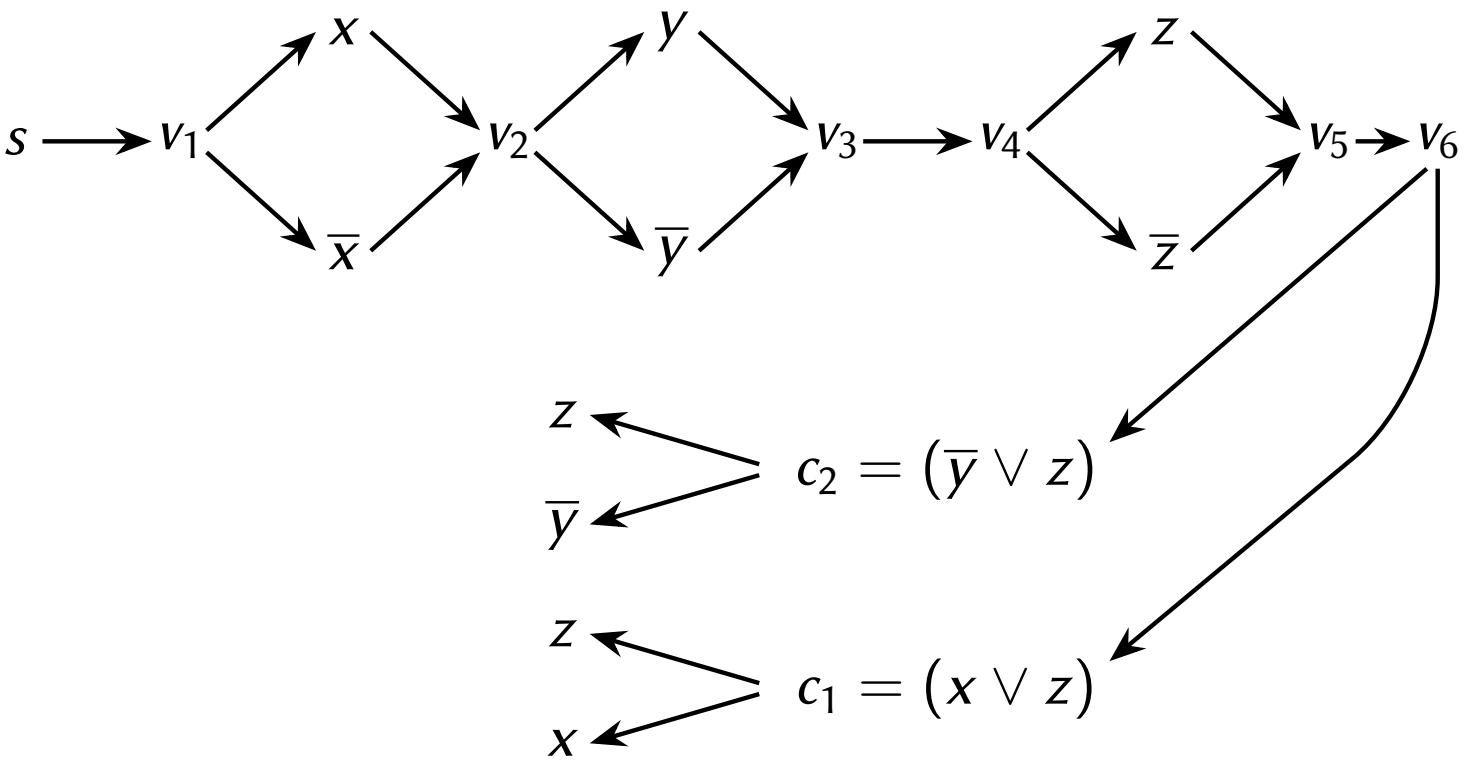


What about the rest of the formula?

The variable choices have been made, now the players want to show the whole thing is true (or false).

- We don't actually check the whole formula — we check only one clause, term, literal, and so on.
- If the formula is *(clause 1)* $\wedge$ *(clause 2)* $\wedge \ldots \wedge$ *(clause m)*, the formula is true only if all clauses are true, but false if one is false.
- Player B is the one trying to make everything false, and that's what could be done here in a single check. So player B chooses the clause.
- If the formula is *(term 1)* $\vee$ *(term 2)* $\vee \ldots \vee$ *(term m)*, the formula is false only if all clauses are false, but true if one is true.
- Player A is the one trying to make everything A, and that's what could be done here in a single check. So player A chooses the term.
- If we see a negation, the players switch roles.

The Resulting Graph



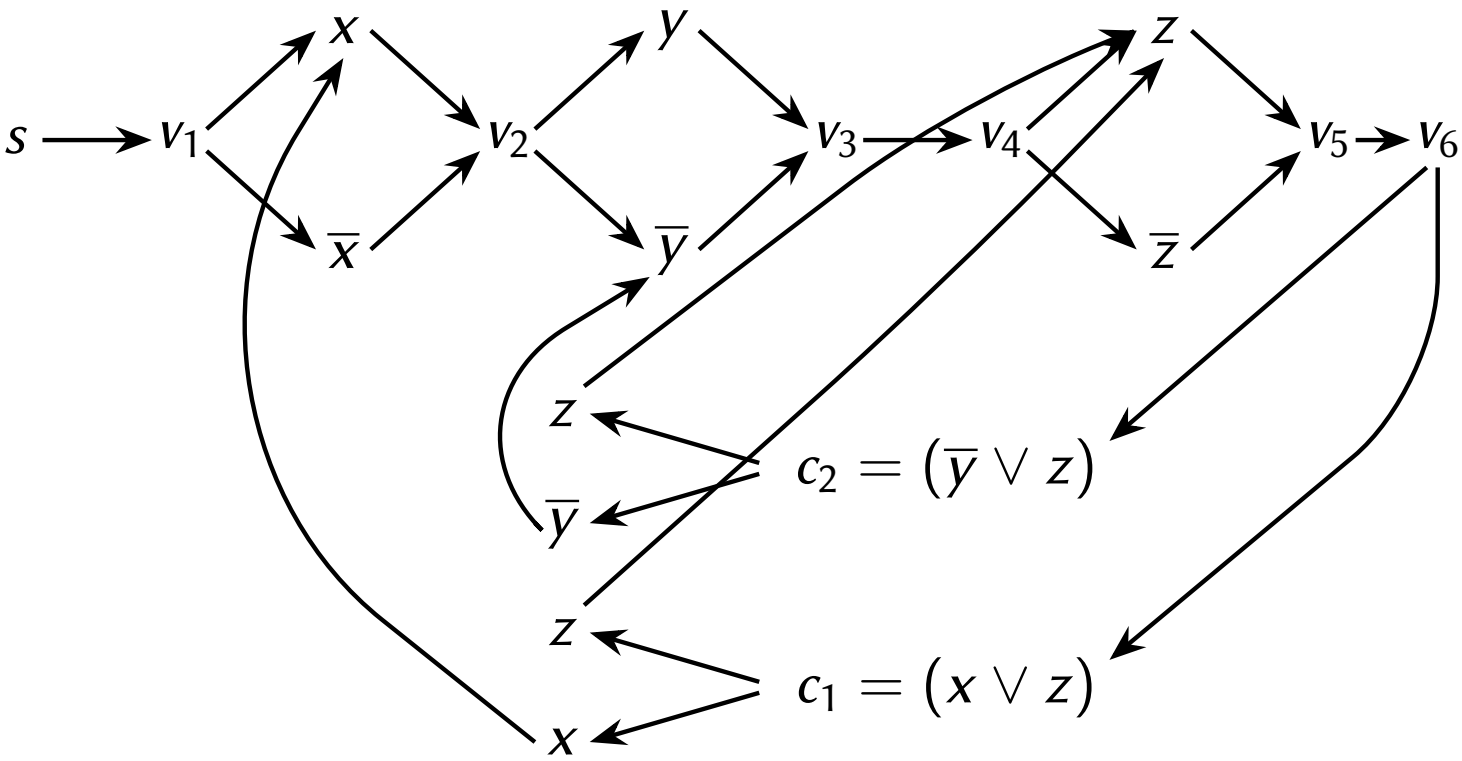
The Resulting Graph (2)

we've added nodes to the graph for every clause and subclause in the formula. If the clauses are joined by *and*s, player B chooses which clause to go to. If they are joined by *or*s, player A chooses. If there is a *not*, the role of the players changes – player A wants to make the clause false. So we put another step in the graph so make the players switch turns.

Finally, we want to finish the game. We should do this in a way such that if player A wins if the final literal is true, and player B wins otherwise (aside from the role switching through *not*s).

We do this by connecting the endpoints of the graph back to the nodes in the variable choices so that the appropriate player wins.

The Final Graph



And that's the reduction! We can see that the same idea will work for any formula ϕ, and that the construction is polytime in ϕ. So TQBF $\leq_p$ GG, and so GG is $\mathcal{PS}$-complete.

Logarithmic Space

The other space complexity class we'll talk about is the class of logarithmic space problems – that is, the class of problems that take an amount of space that's logarithmic in the size of the input.

Since the input already takes $\mathcal{O}(n)$ space, though, we can't include it in the memory being counted. We consider only TMs that have a read-only input tape, a logarithmic sized work tape, and a write-only output tape. (The inputs and outputs have to restricted, otherwise we could cheat and get a linear amount of memory).

Definition: $\mathcal{L} = \{L | L \text{ is decided by a TM that operates in } \mathcal{O}(\log n) \text{ space}\}$.

Definition: $\mathcal{NL} = \{L | L \text{ is decided by an NTM that operates in } \mathcal{O}(\log n) \text{ space}\}$.

Important: this says $\mathcal{O}(\log n)$, *not* $\mathcal{O}(\log^k n)$.

Why do we Care About this Class?

The idea is that we might be running a machine M that is itself small (limited memory), but which has access to a very large database D. In order to access D, M must be able to say where on D it wants to look. So if D has 2^n entries, M must be able to store $\log(2^n) = n$ numbers.

This type of application comes up when, say, M is your phone or laptop and D is the Internet.

It's because we're looking at this sort of application that we also have the restriction that we work in a space that's $\mathcal{O} \log n$ and not, say, $\log^2(n)$. (Also because $\log(n^k) = k \log n$).

Logspace, Continued

This is a very restricted class – there's really not much we know how to do when we can't even copy the input to the working tape. But we can do some things: we can add and multiply numbers, and we can decide simple languages like $\{0^n 1^n | n \in \mathbb{N}\}$.

How to decide $\{0^n 1^n | n \in \mathbb{N}\}$ in logspace: we can't write to the input tape or copy the input. We can count the zeros and ones, since the space needed to remember a number n is logarithmic in n.

On input x:

1. 1. Set a counter c to 0.
2. While the tape head reads a 0:
3. Increment c by 1 and move the head right.
4. While the tape head reads a 1:
5. Decrement c by 1 and move the head right.
6. If c is set to 0 as the end of the input is reached, *accept*. If it reaches 0 before that time, or if it is non-zero at the end of the input, *reject*.

How Do the Classes $\mathcal{L}$ and $\mathcal{NL}$ Relate?

Savitch's Theorem Doesn't Apply Here

Reason: $\mathcal{L}$ uses $\mathcal{O} \log n$ and not $\mathcal{O} \log^2 (n)$ space is that Savitch's theorem doesn't give us a way to emulate non-determinism and still stay in the class. So the best we can currently say is that $\mathcal{L} \subseteq \mathcal{NL}$. We believe that the two classes are not equal.

So what we know is that

$$\mathcal{L} \subseteq \mathcal{NL} \subseteq \mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{PS} \subseteq \mathcal{EXP}.$$

We believe all of the inequalities are strict, but aside from the facts that $\mathcal{L} \neq \mathcal{PS}$ and $\mathcal{P} \neq \mathcal{EXP}$, we don't yet have any proofs. We haven't even proven that 3SAT $\notin \mathcal{NL}$!

Logspace Reductions

Just like polyspace reductions are too powerful for $\mathcal{PS}$ analysis, polytime reductions are too powerful for logspace analysis.

Definition: We say that $A \leqslant_L B$ is there is a function f computable in logspace such that $x \in A$ iff $f(x) \in B$.

If we're working in logspace we won't, in general, even have the space to write $f(x)$. But the point of reductions is to argue that B can be used to solve A. So when testing whether if $f(x)$ is a member of B in order to tell us whether x is in A, we've got to test whether $f(x) \in B$. When we do so, we have to call the function f repeatedly.

Computations in $\mathcal{L}$ and $\mathcal{NL}$ often involve repeating the same calculation over and over in order to save space.

Definition: A language L is $\mathcal{NL}$-complete if it is in $\mathcal{NL}$, and if for every language A in $\mathcal{NL}$, $A \leqslant_L L$.

PATH is $\mathcal{NL}$ (and co-$\mathcal{NL}$)-complete

Our go-to $\mathcal{NL}$-complete language is PATH.

The language PATH is clearly in $\mathcal{NL}$: you can just start at s and non-deterministically follow edges until you reach t. You can halt after checking n vertices, where n is the number of vertices in G.

The proof that $\forall A \in \mathcal{NL}, A \leqslant_L$ PATH is more complicated. Like Savitch's theorem, we won't cover this in detail, and we won't be expecting you to remember this proof, but it is good to know. You can see Sipser 8.25 for details.

Finally, we can show that PATH $\in$co-$\mathcal{NL}$! Again, we won't show the proof, but you can find it in Sipser 8.27.

What this means is that $\mathcal{NL} =$ co-$\mathcal{NL}$.

. . .and that's what we're going to cover about space complexity.

Another Topic: Non-Decision Problems

Computable Functions

Up until now most of the problems we've worked with have been decision problems — problems that return a *yes/no* value.

But there are other types of computations we sometimes want to do — sometimes we want a number or a string or some other more complex data type as a return value, too. How can we talk about these computations?

> E.g., instead of asking "Is $x^2 = y$?" we could ask "What is x^2?".
> Instead of asking "Does the graph G have a path from s to t?" we could ask
>
> - "What is a path from s to t?", or
> - "What is the longest/shortest path from s to t?", or even
> - "How many s to t paths are there?".
>
> Each of these questions ends up being very different in terms of difficulty.

TMs and Functions

As a recap:

- The return value of a TM: whatever is on its tape *(or on its output tape, if it has a designated output tape)* when it halts.

  Practically we just indicate this by adding a return line to our pseudocode.

Functions aren't part of language classes:

- Language classes like $\mathcal{P}$ or $\mathcal{NP}$ are collections of *languages* — of *yes*/*no* questions.
- Functions are not *yes*/*no* questions.
- So problems like: "Given an $x \in \mathbb{N}$, find x^2." isn't in $\mathcal{P}$, even though we can compute it in polytime.

But there are functional analogues to these language classes (e.g., the analogue to the class of decidable languages would be the class of computable functions). We'll look at those classes here.

The Function Class $\mathcal{FP}$

The functional analogue to $\mathcal{P}$ is the class $\mathcal{FP}$ (*Function* $\mathcal{P}$).

The class $\mathcal{FP}$ contains questions like

- "Given an $x \in \mathbb{N}$, what is x^2?",
- "Given a graph G and an s and t, find a path in G from s to t.", or
- "Given a graph G and an s and t, find the shortest path in G from s to t.".

It also contains all of our polytime reductions!
Can you see why?

An analogue for $\mathcal{NP}$?

what would we do for an $\mathcal{NP}$ analogue?

Consider the following function:

FIND-HAM-PATH:

> **Input:** A graph G with specified vertices s and t.
>
> **Output:** Find a Hamiltonian path in G from s to t. Return *null* if no such path exists.

This is a related problem, and it's computationally equivalent to the HAM-PATH problem (if we can solve one we can solve the other).

- Can we use something like this for our $\mathcal{NP}$-analogue?

An analogue for $\mathcal{NP}$? (2)

A language L is in $\mathcal{NP}$ if we can describe it in terms of its certificates — if there's some polytime verifier V such that

$$L = \left\{ x \,\middle|\, \exists c, V \text{ accepts } \langle x, c \rangle \right\}.$$

a very natural $\mathcal{NP}$-analogue question would be: "Given an x, find one of its certificates c (or *null* if no certificate exists)".

We refer to the types of problems that can be expressed this way as $\mathcal{FNP}$. This class is a close analogue to $\mathcal{NP}$: in particular:

$\mathcal{FP} \subseteq \mathcal{FNP}$, and we strongly suspect that these classes are different.

In fact, $\mathcal{P} = \mathcal{NP}$ *iff* $\mathcal{FP} = \mathcal{FNP}$.

The Class $\mathcal{FNP}$

There is a very interesting fact about $\mathcal{FNP}$ to remember:

Suppose we take some language L that is in $\mathcal{NP}$. This L will have an associated $\mathcal{FNP}$ problem f_L. If L is, in fact, $\mathcal{NP}$-complete, it will always be the case that we can use a solver for L to solve f_L, as well. We say that L is *self-reducible*.

> *Unfortunately, we only know that this works for the $\mathcal{NP}$-complete problems. It is possible that there are $\mathcal{NPI}$ problems for which this property does not hold.*

The Class $\mathcal{FNP}$ (2)

It's not quite a perfect $\mathcal{NP}$ analogue, thouch ...

- The idea also works for co-$\mathcal{NP}$-complete languages, and can even help decide some (but not all) languages that we think are in neither $\mathcal{NP}$ nor co-$\mathcal{NP}$.

  Question 6 from assignment 3 would be an example.

But a graph can have more than one Hamiltonian path! What then?

- Any function that reliably gives us any one of the Hamiltonian paths in a graph, or *null* if none exists, would be in $\mathcal{FNP}$.

Downward Self-Reducibility

We've said that we can use a decider (or an oracle) for an $\mathcal{NP}$-complete problem to find one of its certificates. How can we do this?

Suppose we had an oracle **HP** that could solve the decision problem HAM-PATH for us: given $\langle G, s, t \rangle$, **HP** would return *true* if G has a Hamiltonian path from s to t, and *false* otherwise.

We could then write the following code:
Let $M =$ "On input $\langle G, s, t \rangle$:

1. Run **HP**($\langle G, s, t \rangle$). If it returns false, return null.
2. Let $G' = G$.
3. For every edge e in G':
4. Let $G'' = G'$ with e removed.
5. Run **HP**($\langle G'', s, t \rangle$).
6. If it returns true, set $G' = G''$.
7. Set $v_1 = s$, and use the remaining edges in G' to give an ordering to the vertices

Downward Self-Reducibility (2)

We can do the same thing for 3SAT — suppose we had an oracle **3S** for 3SAT. We could then write the following code:
Let $M =$ "On input $\langle \phi \rangle$:

1. Run **3S**($\langle \phi \rangle$). If it returns false, return null.
2. Let $\phi' = \phi$ and $\tau = \{\}$.
3. For every variable x_i in ϕ':
4. Let $\phi'' = \phi' \wedge (x_i \vee x_i \vee x_i)$.
5. Run **3S**($\langle \phi'' \rangle$).
6. If it returns true, set $\phi' = \phi''$ and $\tau = \tau \cup \{x_i : \text{TRUE}\}$.
7. Otherwise, set $\phi' = \phi' \wedge (\overline{x_i} \vee \overline{x_i} \vee \overline{x_i})$ and $\tau = \tau \cup \{x_i : \text{FALSE}\}$.
8. Return τ."

Downward Self-Reducibility (3)

When you answer these problems, remember:

- These are algorithms you're writing, and in this course that means you have to give a short justification.
- You need to show they're in polytime, given their oracles. So you'll give a run-time analysis under the assumption that the oracle takes $\mathcal{O}(1)$ time.
- Using the Cook-Levin reduction makes things too easy, so you can't use that for self-reduction questions.

The Cook-Levin Theorem and Self-Reducibility

Suppose that L is $\mathcal{NP}$-complete and that we have an oracle for L.

If $L \in \mathcal{NP}$, it has polytime verifier V.

Consider the NTM

Let $N =$ "On input x:

1. Nondeterministically choose certificate c for x.
2. Deterministically run V on $\langle x, c \rangle$.
3. If it accepts, *accept*. Otherwise *reject*.

The Cook-Levin Theorem and Self-Reducibility (2)

Given an instance $\langle x \rangle$ of L, we use x and N in the Cook-Levin reduction.

Recall that we try to build table of configurations:

#	q_0	x_0	x_1	$\cdots$	x_{n-1}	$\sqcup$	$\sqcup$	$\cdots$	$\sqcup$	#
#	$t_{1,0}$	q_1	x_1	$\cdots$	x_{n-1}	$\sqcup$	$\sqcup$	$\cdots$	$\sqcup$	#
#										#
#										#
$\vdots$	$\vdots$	$\vdots$				$\vdots$				$\vdots$
#	$t_{n^k-1,0}$	$t_{n^k-1,1}$			$\cdots$				t_{n^k-1,n^k-1}	#

The Cook-Levin Theorem and Self-Reducibility (3)

We build our ϕ to encode this table:

- The variables (mostly) tell us what characters are in the cells.
- The clauses enforce constraints so that the table indeed gives a configuration history.

What this means:

- We can use a satisfying truth assignment τ to fill the table.
- Once we have filled the table, we can read off the certificate c for x (since we're using an N that explicitly writes c).

How to get a satisfying truth assignment τ?

- Use our 3SAT self-reduction!
- When we need the 3SAT oracle, use the 3SAT $\leqslant_p L$ reduction and the L oracle instead.